

Knapsack distribuito: implementazione, comparazione e analisi

Matteo Ielacqua

1 Introduzione

L'algoritmo dello zaino è un noto problema di ottimizzazione. L'idea è che si ha a disposizione uno zaino e diversi elementi ad ognuno dei quali è assegnato un peso e un profitto. L'obiettivo è di massimizzare il profitto mantenendo il peso totale degli elementi entro un limite prefissato. Nella versione detta 0/1, ogni elemento può essere preso una sola volta, il problema si pone quindi come

$$\max \sum v_i x_i$$

dove v_i è il profitto dell'oggetto i -esimo e x_i rappresenta la scelta (1 se scelto 0 altrimenti), soggetto a

$$\sum w_i x_i \leq W \wedge x_i \in \{0, 1\}$$

Le versioni più complete di questo problema prevedono che si possa scegliere lo stesso elemento più volte, sia con che senza limite superiore a queste scelte (rispettivamente Bounded o Unbounded). Esistono anche versioni più complesse, come lo zaino multidimensionale, dove non esiste un solo vincolo (come nel caso più semplice il peso) ma esistono diversi vincoli da tenere in considerazione durante il calcolo. Questa classe di problemi gioca un ruolo chiave in diversi campi molto delicati ad esempio nelle operazioni logistiche o nello scheduling di task satellitari, in cui infatti stanno prendendo piede diverse soluzioni basate su euristiche.

1.1 Soluzioni per knapsack 0/1

Il problema è NP-completo, data una soluzione non esiste quindi un algoritmo che possa verificare se è ottima in tempo polinomiale. Esiste tuttavia una soluzione che lo risolve in tempo pseudo-polinomiale basata sulla programmazione dinamica. La prima soluzione più semplice e meno ottimale ha complessità temporale $O(2^n)$ ed è basata su un albero decisionale esplorato ricorsivamente

Algorithm 1: Knapsack 0/1 Ricorsivo

```
1: profits, weights, max_weight, index
2: Profitto massimo ottenibile
3: if index = 0 or max_weight = 0 then
4:   return 0
5: end
6: pick ← 0
7: if weights[index - 1] ≤ max_weight then
```

```

8:      profits[index - 1] + (
          sequence(
            smallcaps(body: [Knapsack]),
            [()],
            sequence(
              equation(
                block: false,
                body: sequence(
                  [max_weight],
                  [ ],
                  [-],
                  [ ],
                  sequence(
                    [weights],
                    lr(
                      body: sequence([[]], [index], [ ], [-], [ ], [1], [[]]),
                    ),
                  ),
                ),
              [ ],
              [ ],
            equation(block: false, body: [profits]),
            [ ],
            [ ],
            equation(block: false, body: [weights]),
            [ ],
            [ ],
            equation(
              block: false,
              body: sequence([index], [ ], [-], [ ], [1]),
            ),
            [ ]),
        ),
    pick ← )
9: end

```

```

10:         (
            sequence(
                smallcaps(body: [Knapsack]),
                [()],
                sequence(
                    equation(block: false, body: [max_weight]),
                    [()],
                    [ ],
                    equation(block: false, body: [profits]),
                    [()],
                    [ ],
                    equation(block: false, body: [weights]),
                    [()],
                    [ ],
                    equation(
                        block: false,
                        body: sequence([index], [ ], [-], [ ], [1]),
                    ),
                ),
                [()],
            ),
            not_pick ← )
11: return op(«max»)(pick, not_pick)

```

Le altre due soluzioni sono basate sulla programmazione dinamica. la prima usa un approccio top-down (memoization) proponendo sostanzialmente una naturale evoluzione dell'approccio precedente, ha una complessità temporale e spaziale $O(n * W)$

Algorithm 2: Knapsack 0/1 Top-Down (memoization)

```

1: profits, weights, max_weight, index, table
2: Profitto massimo ottenibile
3: if index = 0 or max_weight = 0 then
4:     return 0
5: end
6: if table[index][max_weight] ≠ -1 then
7:     return table[index][max_weight]
8: end
9: pick ← 0
10: if weights[index - 1] ≤ max_weight then

```

```

11:         profits[index - 1] + (
            sequence(
                smallcaps(body: [Knapsack]),
                [()],
                sequence(
                    equation(
                        block: false,
                        body: sequence(
                            [max_weight],
                            [ ],
                            [-],
                            [ ],
                            sequence(
                                [weights],
                                lr(
                                    body: sequence([[]], [index], [ ], [-], [ ], [1], [[]]),
                                ),
                            ),
                        ),
                    ),
                    [ ],
                    [ ],
                    equation(block: false, body: [profits]),
                    [ ],
                    [ ],
                    equation(block: false, body: [weights]),
                    [ ],
                    [ ],
                    equation(
                        block: false,
                        body: sequence([index], [ ], [-], [ ], [1]),
                    ),
                    [ ],
                    [ ],
                    equation(block: false, body: [table]),
                ),
            [ ]),
        pick ← )
12: end

```

```

13:         (
            sequence(
                smallcaps(body: [Knapsack]),
                [(],
                sequence(
                    equation(block: false, body: [max_weight]),
                    [.,],
                    [ ],
                    equation(block: false, body: [profits]),
                    [.,],
                    [ ],
                    equation(block: false, body: [weights]),
                    [.,],
                    [ ],
                    equation(
                        block: false,
                        body: sequence([index], [ ], [-], [ ], [1]),
                    ),
                    [.,],
                    [ ],
                    equation(block: false, body: [table]),
                ),
                [)],
            ),
        not_pick ← )
14: table[index][max_weight] ← op(«max»)(pick, not_pick)
15: return table[index][max_weight]

```

la seconda è una versione bottom-up (tabulation) con la medesima complessità ma che evita la ricor-
sione

Algorithm 3: Knapsack 0/1 Bottom-Up (tabulation)

```

1: profits, weights, max_weight, len
2: Profitto massimo ottenibile
3: table ← Call[Matrix][len + 1, max_weight + 1]
4: for i ← 0 to len do
5:     for j ← 0 to max_weight do
6:         if i = 0 or j = 0 then
7:             table[i][j] ← 0
8:             pick ← 0
9:             if weights[i - 1] ≤ j then
10:                pick ← profits[i - 1] + table[i - 1][j - weights[i - 1]]
11:             end
12:             not_pick ← table[i - 1][j]
13:             table[i][j] ← op(«max»)(pick, not_pick)
14:         end
15:     end
16: end
17: return table[len][max_weight]

```

Una soluzione più raffinata di questa prevede di usare solo l'ultima riga calcolata, in modo da ridurre la complessità spaziale a $O(W)$

Algorithm 4: Knapsack 0/1 Bottom-Up (spazio ottimizzato)

```
1: profits, weights, max_weight, len
2: Profitto massimo ottenibile
3: table  $\leftarrow$  Call[array][0... max_weight] inizializzato a 0
4: for i  $\leftarrow$  1 to len do
5:   for j  $\leftarrow$  max_weight downto weights[i - 1] do
6:     table[j]  $\leftarrow$  op(«max»)(table[j], table[j - weights[i - 1]] + profits[i - 1])
7:   end
8: end
9: return table[max_weight]
```

Lo svantaggio principale delle soluzioni basate su programmazione dinamica è che non lavorano bene su casi in cui i pesi non sono interi (o non possiedono una differenza prefissata tra di loro). Lo stesso vale per l'approccio naive descritto nella prima soluzione.

1.2 Soluzione Greedy

La soluzione Greedy ha un indubbio vantaggio rispetto a tutte le altre, può scegliere anche frazioni di un oggetto, l'idea è quella di prendere gli oggetti basandosi sul loro rapporto peso/valore, prendendoli in ordine decrescente di rapporto, finché lo zaino non è pieno. Se un oggetto non è possibile prenderlo per intero allora si può prendere una sua frazione a seconda della capacità rimanente.

Algorithm 5: Fractional Knapsack (Greedy)

```
1: profits, weights, max_weight, len
2: Profitto massimo ottenibile
3: items  $\leftarrow$  Call[sort][Call[zip][profits, weights], key =  $\frac{\text{profit}}{\text{weight}}$  decrescente]
4: profit  $\leftarrow$  0.0
5: current_capacity  $\leftarrow$  max_weight
6: for i  $\leftarrow$  0 to len - 1 do
7:   if items[i] · weight  $\leq$  current_capacity then
8:     profit  $\leftarrow$  profit + items[i] · profit
9:     current_capacity  $\leftarrow$  current_capacity - items[i] · weight
10:    profit  $\leftarrow$  profit +  $\frac{\text{items}[i] \cdot \text{profit}}{\text{items}[i] \cdot \text{weight}} \times \text{current\_capacity}$ 
11:    BREAK
12:   end
13: end
14: return profit
```

2 Soluzioni parallele al Knapsack 0/1

Le soluzioni a questo problema sono le più disparate e negli anni ne sono state scritte molte. In ambito aereospaziale il knapsack è usato soprattutto per lo scheduling di task operativi satellitari [1] in particolare per l'osservazione terrestre [2]. Un interessante modello di esecuzione è denominato COPA [3] e sviluppa un algoritmo parallelo cost optimal per risolvere il problema sia su CPU con OpenMP che su GPU con CUDA, l'idea dietro quest'algoritmo è la seguente: 1) Si prende l'array di oggetti V e lo si divide in 2 insiemi disgiunti. 2) Per il primo insieme si generano tutte le $N = 2^{n/2}$ possibili combinazioni a_i , si calcola di ogni sottoinsieme a_i peso e profitto w_i, p_i e si aggiunge questa tripletta in una lista in ordine crescente. 3) Si esegue lo stesso con la seconda lista per ogni sottoinsieme b_i e le triplette si mettono in una lista di ordine non crescente. In questo modo si è generata la lista A, per generare la lista B si riproducono gli step precedenti semplicemente inserendo le triplette in una lista

di ordine non crescente. A questo punto si può passare alla fase di ricerca che prenderà in input le due liste ordinate.

Algorithm 6: Algoritmo di ricerca

```

1: Due liste ordinate  $A$  e  $B$ 
2: Soluzione finale Bestvalue e  $X$ 
3: Inizializza  $\text{MaxB}_N \leftarrow b_N \cdot p$ ,  $L_N \leftarrow N$ ,  $\text{Bestvalue} \leftarrow 0$ ,  $X_1 \leftarrow (0, 0)$ 
4: for  $i \leftarrow N - 1$  downto  $1$  do
5:   if  $b_i \cdot p > \text{MaxB}_{i+1}$  then
6:      $\text{MaxB}_i \leftarrow b_i \cdot p$  and  $L_i \leftarrow i$ 
7:      $\text{MaxB}_i \leftarrow \text{MaxB}_{i+1}$  and  $L_i \leftarrow L_{i+1}$ 
8:   end
9: end
10:  $i \leftarrow 1$  and  $j \leftarrow 1$ 
11: while  $i \leq N$  and  $j \leq N$  do
12:   if  $a_i \cdot w + b_j \cdot w > c$  then
13:      $j \leftarrow j + 1$  and continue
14:   end
15:   if  $a_i \cdot p + \text{MaxB}_j > \text{Bestvalue}$  then
16:      $\text{Bestvalue} \leftarrow a_i \cdot p + \text{MaxB}_j$  and  $X_1 \leftarrow (a_i, b_{L_j})$ 
17:   end
18:    $i \leftarrow i + 1$ 
19: end
20: Converti le due componenti decimali di  $X_1$  in due numeri binari:  $X_1 \leftarrow (\text{binary}_1, \text{binary}_2)$ 
21:  $X \leftarrow \text{Call}[\text{«strcat»}][\text{binary}_1, \text{binary}_2]$  ▷ Concatena i due binari

```

Questo modello di esecuzione si può vedere subito che si presta bene ad essere parallelizzato, sia nella parte di generazione delle liste sia in quella poi di ricerca dell'ottimo.

2.1 COPA: Cost Optimal Parallel Algorithm

L'algoritmo parallelo è stato pensato per essere risolto su un modello EREW (Exclusive Read, Exclusive Write) PRAM con shared memory e assume k processori accessibili. L'algoritmo sfrutta prima un'algoritmo di generazione parallela delle liste A e B

Algorithm 7: Algoritmo parallelo di generazione

```

1: Elementi  $(v_i, v_i \cdot w, v_i \cdot p)$ ,  $i = 1, 2, \dots, \frac{n}{2}$ 
2: Lista ordinata  $A_{\frac{n}{2}}$  (i.e.  $A$ )
3: Inizializza  $A_0 \leftarrow [(0, 0, 0)]$  e  $\text{IC} \leftarrow 1$ 
4: for  $i \leftarrow 0$  to  $\frac{n}{2} - 1$  do
5:    $v_{i+1} \leftarrow \text{IC}$ 
6:   for all  $P_j$ ,  $1 \leq j \leq k$ , in parallel do
7:     Genera nuova lista  $A'_i$  aggiungendo la tripletta  $(v_{i+1}, v_{i+1} \cdot w, v_{i+1} \cdot p)$  a tutti gli elementi della lista ordinata  $A_i$ 
8:     Chiama l'algoritmo di merge parallelo per unire  $A_i$  e  $A'_i$  in ordine non decrescente di peso, producendo la lista ordinata  $A_{i+1}$ 
9:   end
10:   $\text{IC} \leftarrow \text{IC} + 1$ 
11: end

```

l'algoritmo di merging parallelo si articola nel modo seguente: prende in input due vettori ordinati $U = (u_1, u_2, \dots, u_m)$ e $V = (v_1, v_2, \dots, v_m)$ e k processori $P_1, P_2, \dots, P_k$, con $1 \leq k \leq 2m$ potenza di 2, e produce un vettore ordinato di lunghezza $2m$ ottenuto dall'unione di U e V .

1. **Step 1:** I k processori partizionano U e V in parallelo, ciascuno in k sottovettori (che possono essere vuoti) $(U_1, U_2, \dots, U_k)$ e $(V_1, V_2, \dots, V_k)$ tali che: $|U_i| + |V_i| = 2m/k$ per $1 \leq i \leq k$, e i pesi di tutti gli elementi in U_i e V_i sono minori dei pesi di tutti gli elementi in U_{i+1} e V_{i+1} , per $1 \leq i \leq k-1$.
2. **Step 2:** Il processore P_i esegue il merge sequenziale di U_i e V_i , per $1 \leq i \leq k$, in parallelo con gli altri processori.

A questo punto si può passare a dividere equamente tra i processori disponibili i blocchi ottenuti e calcolare per ognuno di questi blocchi gli elementi di profitto massimo.

Algorithm 8: Algoritmo di divisione e ricerca del massimo profitto per blocco

```

1: Due liste ordinate  $A$  e  $B$ 
2: MaxA e MaxB
3: Dividi sia  $A$  che  $B$  uniformemente in  $k$  blocchi
4: for all  $P_i$ ,  $1 \leq i \leq k$ , in parallel do
5:   MaxA $i$   $\leftarrow a_{i,1} \cdot p$ , MaxB $i$   $\leftarrow b_{i,1} \cdot p$ 
6:   for  $j \leftarrow 2$  to  $e$  do
7:     if  $a_{i,j} \cdot p > \text{MaxA}_i$  then
8:       MaxA $i$   $\leftarrow a_{i,j} \cdot p$ 
9:     end
10:    if  $b_{i,j} \cdot p > \text{MaxB}_i$  then
11:      MaxB $i$   $\leftarrow b_{i,j} \cdot p$ 
12:    end
13:  end
14: end

```

Ora lo spazio delle soluzioni è davvero molto grande, per questo motivo si ricorre a un algoritmo di potatura dell'insieme delle soluzioni

Algorithm 9: Algoritmo di pruning parallelo

```

1: Due liste di blocchi  $A = \{A_1, A_2, \dots, A_k\}$  e  $B = \{B_1, B_2, \dots, B_k\}$ , capacità  $c$ 
2: Insieme di coppie di blocchi non potate remaining
3: Inizializza remaining  $\leftarrow \emptyset$ 
4: for all  $P_i$ ,  $1 \leq i \leq k$ , in parallel do
5:   Inizializza local_results  $\leftarrow \emptyset$ 
6:   for  $j \leftarrow i$  to  $k+i-1$  do
7:     b_idx  $\leftarrow j \bmod k$ 
8:      $Z \leftarrow A_i \cdot \text{front} \cdot w + B_{b\_idx} \cdot \text{back} \cdot w$ 
9:      $Y \leftarrow A_i \cdot \text{back} \cdot w + B_{b\_idx} \cdot \text{front} \cdot w$ 
10:    if  $Y \leq c$  then
11:       $\triangleright$  Tutte le coppie nel blocco sono valide
12:      if  $A_i \cdot \text{maxValue} + B_{b\_idx} \cdot \text{maxValue} > \text{best\_value}$  then
13:        best_value  $\leftarrow A_i \cdot \text{maxValue} + B_{b\_idx} \cdot \text{maxValue}$ 
14:      end
15:      local_results  $\cdot \text{add}(A_i, B_{b\_idx})$ 
16:    end
17:    else if  $Z \leq c$  and  $Y > c$  then
18:       $\triangleright$  Alcune coppie potrebbero essere valide
19:      local_results  $\cdot \text{add}(A_i, B_{b\_idx})$ 
20:       $\triangleright Z > c$ : nessuna coppia valida, pota il blocco
21:    skip
22:  end
23: end
24: end
25: remaining  $\leftarrow \bigcup_{i=1}^k \text{local\_results}_i$ 

```

L'algoritmo di pruning sfrutta il fatto che i blocchi sono ordinati per peso. Per ogni coppia di blocchi (A_i, B_j) , vengono calcolati due valori chiave:

- $Z = A_i.\text{front}.w + B_j.\text{back}.w$: il peso minimo possibile combinando il primo elemento di A_i con l'ultimo di B_j
- $Y = A_i.\text{back}.w + B_j.\text{front}.w$: il peso massimo possibile combinando l'ultimo elemento di A_i con il primo di B_j

La logica di potatura è la seguente:

1. Se $Y \leq c$, tutte le combinazioni tra i due blocchi rispettano il vincolo di capacità. La coppia di blocchi viene mantenuta e il loro massimo profitto combinato viene aggiornato.
2. Se $Z \leq c < Y$, alcune combinazioni potrebbero essere valide mentre altre no. La coppia di blocchi viene mantenuta per un'analisi più approfondita.
3. Se $Z > c$, nessuna combinazione tra i due blocchi può rispettare il vincolo di capacità. La coppia viene scartata (potata).

A questo punto è possibile passare alla ricerca parallela della soluzione. L'algoritmo si compone di due sotto-fasi: il secondo salvataggio parallelo dei massimi (Algoritmo 5 del paper COPA) e la ricerca parallela vera e propria (Algoritmo 6).

2.1.1 Secondo salvataggio parallelo dei massimi

Dopo il pruning, al più $2k - 1$ coppie di blocchi vengono distribuite equamente tra i k processori: ogni processore P_i gestisce al massimo $s \leq 2$ coppie. Per ogni blocco B_{it} assegnato, si calcola un array di **suffix max**: $\text{Max}_{i:j}[t]$ rappresenta il profitto massimo tra tutti gli elementi da posizione j a e del blocco B_{it} , mentre L_j^t rappresenta l'indice del massimo trovato.

Algorithm 10: Secondo salvataggio parallelo dei massimi (suffix max)

```

1: Coppie di blocchi rimanenti dopo il pruning
2:  $\text{Max}_{i:j}[t]$  e  $L_j^t$  per ogni blocco  $B_{it}$ 
3: for all  $P_i$ ,  $1 \leq i \leq k$ , in parallel do
4:   for  $t \leftarrow 0$  to  $s - 1$  do
5:      $\text{Max}_{i:e}[t] \leftarrow b_{it:e} \cdot p$ ,  $L_e^t \leftarrow e$ 
6:     for  $j \leftarrow e - 1$  downto  $1$  do
7:       if  $b_{it:j} \cdot p > \text{Max}_{i:(j+1)}[t]$  then
8:          $\text{Max}_{i:j}[t] \leftarrow b_{it:j} \cdot p$ ,  $L_j^t \leftarrow j$ 
9:          $\text{Max}_{i:j}[t] \leftarrow \text{Max}_{i:(j+1)}[t]$ ,  $L_j^t \leftarrow L_{j+1}^t$ 
10:      end
11:    end
12:  end
13: end

```

Questo pre-calcolo consente, durante la ricerca, di conoscere in $O(1)$ il miglior profitto ottenibile da un dato punto in poi nella lista B , accelerando notevolmente la scansione.

2.1.2 Ricerca parallela con two-pointer

Ciascun processore esegue una ricerca **two-pointer** sulle proprie coppie di blocchi (A_i, B_{it}) . Per ogni coppia, due puntatori x (su A_i) e y (su B_{it}) scorrono i blocchi in direzioni opposte:

Algorithm 11: Ricerca parallela (two-pointer search)

```

1: Coppie di blocchi rimanenti dopo il pruning, suffix max arrays
2: Soluzione finale Bestvalue e  $X$ 
3: for all  $P_i$ ,  $1 \leq i \leq k$ , in parallel do
4:    $t \leftarrow 0$ ,  $\text{Maxvalue}_i \leftarrow 0$ ,  $X_i \leftarrow (0, 0)$ 

```

```

5:   while  $0 \leq t \leq s - 1$  do
6:     Seleziona la coppia di blocchi  $(A_i, B_{it})$ 
7:      $x \leftarrow 1, y \leftarrow 1$ 
8:     while  $x \leq e$  and  $y \leq e$  do
9:       if  $a_{i:x} \cdot w + b_{it:y} \cdot w > c$  then
10:         $y \leftarrow y + 1$  and continue
11:       end
12:       if  $a_{i:x} \cdot p + \text{Max}_{i:y}[t] > \text{Maxvalue}_i$  then
13:         $\text{Maxvalue}_i \leftarrow a_{i:x} \cdot p + \text{Max}_{i:y}[t]$ 
14:         $X_i \leftarrow (a_{i:x}, b_{it:L_y^t})$ 
15:       end
16:        $x \leftarrow x + 1$ 
17:     end
18:      $t \leftarrow t + 1$ 
19:   end
20: end
21: Riduzione globale:  $\text{Bestvalue} \leftarrow \text{Maxvalue}_1, X \leftarrow X_1$ 
22: for  $i \leftarrow 2$  to  $k$  do
23:   if  $\text{Bestvalue} < \text{Maxvalue}_i$  then
24:     $\text{Bestvalue} \leftarrow \text{Maxvalue}_i, X \leftarrow X_i$ 
25:   end
26: end
27: Converti le componenti di  $X$  in binario e concatena per ottenere la soluzione finale

```

La logica è la seguente: poiché A_i è ordinato per peso crescente e B_{it} per peso decrescente, quando la somma dei pesi supera la capacità c , si incrementa y per passare a un elemento più leggero in B . Quando invece la somma è ammissibile, il profitto candidato si calcola come $a_{i:x} \cdot p + \text{Max}_{i:y}[t]$, dove $\text{Max}_{i:y}[t]$ rappresenta il miglior profitto ottenibile da y in poi nel blocco B_{it} . Questo evita di dover iterare su tutte le coppie di elementi, riducendo la complessità a $O(e) = O(\frac{N}{k})$.

La complessità complessiva dello stage di ricerca parallela è $O(4N/k + k)$, che nel caso $k = O(N^{1/2})$ diventa $O(2^{n/4})$.

3 Implementazione dell'esperimento

l'esperimento si divide in 4 parti distinte: 1) l'implementazione del algoritmo di risoluzione del knapsack con programmazione dinamica: questo metodo è quello più semplice da implementare ed è stato usato come base per verificare la corretta implementazione negli unit test degli algoritmi successivi. 2) Implementazione dell'algoritmo con programmazione dinamica in MPI 3) Implementazione dell'algoritmo COPA sequenziale e parallelo in shared memory usando OpenMP. 4) Implementazione dell'algoritmo COPA distribuito usando MPI

3.1 Implementazione del knapsack in dynamic programming

L'implementazione del knapsack in programmazione dinamica sequenziale è decisamente semplice sia nella versione sequenziale che parallela in shared memory:

```

1  int n = weights.size();
2  std::vector<std::vector<int>> dp(n + 1, std::vector<int>(capacity + 1, 0));
3  // Build the dp table
4  for (int i = 1; i <= n; ++i) {
5      #pragma omp parallel for num_threads(numThreads) // Adjust the number of
        threads as needed
6      for (int w = 0; w <= capacity; ++w) {
7          if (weights[i - 1] <= w) {
8              dp[i][w] = std::max(dp[i - 1][w], dp[i - 1][w - weights[i - 1]] +
values[i - 1]);
9          } else {
10             dp[i][w] = dp[i - 1][w];
11         }
12     }
13 }

```

Listato 1: Knapsack dynamic programming sequenziale

Questo algoritmo ha una complessità di $O(N * C)$ dove C è la capacità dello zaino, la parallelizzazione avviene scomponendo il ciclo for interno che scrive i valori nella tabella, non è possibile invece parallelizzare il ciclo for più esterno perchè per risolvere la linea successiva bisogna prima calcolare tutti i pesi di quella precedente.

3.1.1 Implementazione MPI

L'implementazione MPI dell'algoritmo è molto simile, l'approccio seguito è stato quello di assegnare la tabella a un master, che scompone la linea successiva da calcolare in task di equa lunghezza da distribuire ai vari worker.

```

1  using namespace boost::mpi;
2  int n = static_cast<int>(weights.size());
3  std::vector<uint32_t> line(capacity + 1, 0);
4  for (int i = 1; i <= n; ++i)
5  {
6      broadcast(comm, line.data(), static_cast<int>(line.size()), 0);
7      while (true)
8      {
9          NodeTask task{};
10         auto status = comm.recv(any_source, any_tag, task);
11         if (status.tag() == NODETAG::TERMINATE)
12         {
13             break;
14         }
15         int len = task.endIndex - task.startIndex + 1;
16         std::vector<uint32_t> result(len);
17         for (int w = task.startIndex; w <= task.endIndex; ++w)
18         {
19             if (weights[i - 1] <= w)
20                 result[w - task.startIndex] = std::max(line[w], line[w -
weights[i - 1]] + values[i - 1]);
21             else
22                 result[w - task.startIndex] = line[w];
23         }
24         comm.send(status.source(), NODETAG::RESPONSE,
25         NodeResponse{task.startIndex, task.endIndex});
26         comm.send(status.source(), NODETAG::DATA,
27         NodeResponseData{std::move(result)});
28     }
29     comm.barrier();
30 }

```

Listato 2: Worker task

i task sono dei range di indici che il worker deve computare, una volta finito il worker invia al master il blocco appena computato dopodichè il worker attende tutti i nodi alla barriera e si comincia il calcolo per il prossimo oggetto.

```

1  int n = static_cast<int>(weights.size());
2  std::vector<std::vector<uint32_t>> dp(n + 1, std::vector<uint32_t>(capacity + 1,
3  0));
4  for (int i = 1; i <= n; ++i)
5  {
6      broadcast(MPI_COMM_WORLD, dp[i - 1].data(), capacity + 1, 0); //broadcast the
7      previous line from master
8      int actual_index = 0;
9      int active = 0;
10     auto generator = [capacity, chunk_size, &actual_index, &active]() ->
11     std::optional<NodeTask> {
12         if (actual_index > capacity)
13         {
14             return std::nullopt;
15         }
16         int start = actual_index;
17         int end = std::min(actual_index + chunk_size - 1, capacity);
18         actual_index = end + 1;
19         active++;
20         return NodeTask{start, end};
21     };
22     send_a_task_to_all_workers(comm, generator);
23     while(active > 0)
24     {
25         auto result = receive_a_result_from_any_worker(comm);
26         active--;
27         copy_result_to_dp_table(dp[i], result);
28     }
29     terminate_all_workers(comm);
30     comm.barrier();
31 }

```

Listato 3: Master Task

terminati i cicli il master può leggere dalla DP il risultato. Si evince subito che l'implementazione MPI è piuttosto semplice e segue uno schema molto simile alla versione Shared Memory, tuttavia presenta lo svantaggio di dover comunicare la linea calcolata a tutti i worker. Svantaggio che si avrebbe comunque anche nel caso della versione più ottimizzata, in cui non esiste la DP perchè non si esegue il backtracking per ricostruire gli oggetti scelti, ma si calcola solo il profitto massimo ottenibile. In questo caso infatti, per calcolare la linea successiva della DP è necessario conoscere la linea precedente, quindi anche in questo caso si dovrebbe comunicare a tutti i worker la linea precedentemente calcolata.

3.2 Knapsack COPA

La versione COPA dell'algoritmo è stata pensata per essere eseguita in shared memory e per ovviare a un problema della versione in dynamic programming dell'algoritmo, cioè che la grandezza della tabella cresce con l'aumentare della capacità dello zaino. Invece nella versione COPA il numero di soluzioni da esplorare è perlopiù legato al numero degli elementi, tuttavia questo insieme cresce con un ritmo di $2^{n/2}$, quindi per un numero di elementi superiore a 40-50 il numero di combinazioni diventa proibitivo da memorizzare (come anche sottolineato nell'articolo originale).

3.2.1 Algoritmo di generazione dei sottoinsiemi

I sottoinsiemi sono generati partendo dalla lista generale, dividendola in due e generando a partire dalle due liste spezzate un insieme di sottoinsiemi di soluzioni. L'algoritmo si articola in due fasi: 1) copia della lista precedente e aggiunta dell'elemento a ogni insieme 2) merging della nuova lista con quella precedente usando l'algoritmo di coranking.

```

1  using namespace std::ranges;
2  int total_size = static_cast<int>(A.size() + B.size());
3
4  #pragma omp parallel for num_threads(num_threads)
5  for (int t = 0; t < num_threads; ++t)
6  {
7      int start = t * total_size / num_threads;
8      int end = (t + 1) * total_size / num_threads;
9
10     auto [a_start, b_start] = co_rank(A, B, start);
11     auto [a_end, b_end] = co_rank(A, B, end);
12     std::merge(begin(A) + a_start, begin(A) + a_end, begin(B) + b_start,
13               begin(B) + b_end, begin(output) + start);
14 }

```

Listato 4: funzione di Parallel Merging

```

1  for (int item_idx = 0; auto &&item : r)
2  {
3      std::vector<CopaSubset> shifted{subsets.size()};
4      auto [w, v] = item;
5      for (int i = 0; i < static_cast<int>(shifted.size()); i++)
6      {
7          shifted[i] = subsets[i];
8          shifted[i].addItem(item_idx, w, v);
9      }
10     item_idx++;
11     std::vector<CopaSubset> merged;
12     merged.resize(subsets.size() + shifted.size());
13     parallel_merge(subsets, shifted, merged, numthreads);
14     subsets = std::move(merged);
15 }

```

Listato 5: Funzione di generazione e merging

Per aumentare il più possibile la località dei dati durante questa fase, si copia il dato e gli si aggiunge l'item direttamente nelle istruzioni successive. Questa funzione nello specifico restituisce il metodo di generazione della lista A in ordine crescente, per la lista B è sufficiente usare `std::reverse`, una possibile ottimizzazione sarebbe quella di usare `std::ranges::reverse` in modo che il reversing dei dati sia lazy, cioè effettuato solo quando si pesca dal range. Tuttavia visto che le funzioni successive richiederanno l'uso di input range, che richiede che il range in entrata sia contiguo, questa operazione non sarà possibile poichè il reverse range non è contiguo per lo standard c++. A questo punto si può procedere e suddividere la lista di blocchi in sottoinsiemi di blocchi equamente distribuiti tra i processori, avendo cura di calcolare il massimo profitto durante questa operazione.

```

1  int n = static_cast<int>(std::ranges::size(input));
2  int k = num_threads;
3  int block_size = n / k;
4  int remainder = n % k;
5  #pragma omp parallel for num_threads(num_threads)
6  for (int i = 0; i < k; ++i)
7  {
8      int start = i * block_size + std::min(i, remainder);
9      int end = start + block_size + (i < remainder ? 1 : 0);
10     if (start >= end)
11     {
12         output[i] = {std::span<const CopaSubset>{}, 0};
13         continue;
14     }
15     auto span = std::ranges::subrange(std::ranges::begin(input) + start,
std::ranges::begin(input) + end);
16     int max_val = 0;
17     for (const auto &elem : span)
18     {
19         if (elem.totalValue > max_val)
20             max_val = elem.totalValue;
21     }
22     output[i] = {span, max_val};
23 }

```

Listato 6: Funzione di calcolo del massimo e distribuzione dei blocchi

A questo punto ci si ritrova in output una lista di sottoinsiemi di blocchi, ogni elemento corrisponde al sottoinsieme su cui ci si aspetta che il processo p operi.

3.2.2 Pruning delle soluzioni

Una volta compiuta l'operazione di distribuzione si può passare a eseguire il pruning delle soluzioni.

```

1  int k = static_cast<int>(blocksB.size());
2  int best_value = 0;
3  for (int j = i; j < k + i; ++j)
4  {
5      int b_idx = j % k;
6      const auto &blockB = blocksB[b_idx];
7      if (blockB.block.empty())
8          continue;
9      int Z = blockA.block.front().totalWeight + blockB.block.back().totalWeight;
10     int Y = blockA.block.back().totalWeight + blockB.block.front().totalWeight;
11
12     // Note: prune is done by not adding the pair to local_results
13     if (Y <= capacity)
14     {
15         // All pairs in this block pair are valid; save max profit and prune
16         if (blockA.maxValue + blockB.maxValue > best_value)
17         {
18             best_value = blockA.maxValue + blockB.maxValue;
19         }
20         remaining.emplace_back(blockA, blockB);
21     }
22     else if (Z <= capacity && Y > capacity)
23     {
24         remaining.emplace_back(blockA, blockB);
25     }
26     else if (Z > capacity)
27     {
28         // No pairs in this block pair are valid; prune
29     }
30 }

```

Listato 7: Funzione di pruning di un sottoinsieme di blocchi

La suddetta funzione deve essere chiamata in modo parallelo in questo modo. La suddivisione è necessaria poichè questa implementazione della funzione di pruning è stata riutilizzata anche nella versione MPI.

```

1  std::vector<std::vector<std::pair<CopaBlock, CopaBlock>>>
   local_results(threads);
2  #pragma omp parallel for num_threads(threads)
3  for (int i = 0; i < threads; ++i)
4  {
5      const auto &blockA = blocksA[i];
6      prune_block_pair(blockA, blocksB, local_results[i], capacity, i);
7  }
8  // Merge local results from all processors
9  for (const auto &local : local_results)
10 {
11     remaining.insert(remaining.end(), local.begin(), local.end());
12 }

```

Listato 8: Chiamata parallela della funzione di pruning

A questo punto si può illustrare la funzione di ricerca parallela dei massimi, che viene chiamata sui blocchi rimanenti in questo modo, anche in questo caso la funzione è stata disaccoppiata poichè la parte di calcolo è stata riutilizzata nella versione a memoria distribuita.


```

1  #pragma omp parallel for num_threads(numThreads)
2  for (int i = 0; i < numThreads; ++i)
3  {
4      const auto &[blockA, blockB] = remainingPairs[i];
5
6      // Stage 5: Two-pointer search within this block pair
7      BlockPairSearchResult result = block_pair_pointer_search(blockA, blockB,
8      capacity);
9      processBestVal[i] = result.bestVal;
10     processBestAIdx[i] = result.bestAIdx;
11     processBestBIdx[i] = result.bestBIdx;
12 }

```

Listato 9: Ricerca parallela dei massimi sui blocchi rimanenti

La funzione prende in input le coppie rimanenti in ciascuna lista, che è un vettore di coppie di blocchi, e li esamina in questo modo: prima calcola il suffisso massimo di profitto tra i blocchi della lista B, dopodichè esegue una comparazione con i pesi della lista A.

```

1  int blocka_size = static_cast<int>(blockA.block.size());
2  int blockb_size = static_cast<int>(blockB.block.size());
3  // Stage 4: Suffix max for this B block
4  std::vector<int> suffixMaxVal(blockb_size);
5  std::vector<int> suffixMaxIdx(blockb_size);
6  block_suffix_max_values(blockB.block, suffixMaxVal, suffixMaxIdx);
7
8  int x = 0, y = 0;
9  int bestVal = 0, bestA = 0, bestB = 0;
10 while (x < blocka_size && y < blockb_size)
11 {
12     if (blockA.block[x].totalWeight + blockB.block[y].totalWeight > capacity)
13     {
14         y++;
15         continue;
16     }
17     int candidate = blockA.block[x].totalValue + suffixMaxVal[y];
18     if (candidate > bestVal)
19     {
20         bestVal = candidate;
21         bestA = blockA.block[x].index;
22         bestB = suffixMaxIdx[y];
23     }
24     x++;
25 }
26 return {bestVal, bestA, bestB};

```

Listato 10: Funzione di ricerca del massimo tra i blocchi rimanenti

La funzione di calcolo dei suffissi è implementata in questo modo

```

1  int e = static_cast<int>(block.size());
2  suffixMaxValues[e - 1] = block[e - 1].totalValue;
3  suffixMaxIndices[e - 1] = block[e - 1].index;
4  for (int j = e - 2; j >= 0; --j)
5  {
6      if (block[j].totalValue > suffixMaxValues[j + 1])
7      {
8          suffixMaxValues[j] = block[j].totalValue;
9          suffixMaxIndices[j] = block[j].index;
10     }
11     else
12     {
13         suffixMaxValues[j] = suffixMaxValues[j + 1];
14         suffixMaxIndices[j] = suffixMaxIndices[j + 1];
15     }
16 }

```

Listato 11: Funzione di calcolo dei suffix max

3.2.3 L'algoritmo completo

La funzione completa si articola quindi in questo modo

```

1  using namespace std::views;
2  auto list = zip(weights, values);
3  int n = static_cast<int>(weights.size());
4  auto Alist = take(list, n / 2);
5  auto Blist = drop(list, n / 2);
6  std::vector<CopaSubset> A, B;
7  generate_copa_subsets(Alist, numThreads);
8  generate_copa_subsets(Blist, numThreads, true);
9  int N = static_cast<int>(B.size());
10 // Stage 2 : Parallel suffix max for B (MaxBj and Lj)
11 std::vector<CopaBlock> blocksA(numThreads);
12 std::vector<CopaBlock> blocksB(numThreads);
13 distribute_block_per_processor(A, blocksA, numThreads);
14 distribute_block_per_processor(B, blocksB, numThreads);
15 // Stage 3: Parallel pruning
16 std::vector<std::pair<CopaBlock, CopaBlock>> remainingPairs;
17 prune(blocksA, blocksB, remainingPairs, capacity, numThreads);
18 // Stages 4+5: For each remaining block pair, compute suffix max and search
19 int k = static_cast<int>(remainingPairs.size());
20 int bestAIdx = 0, bestBIdx = 0, bestValue = 0;
21 std::vector<int> localBestVal(k, 0);
22 std::vector<int> localBestAIdx(k, 0);
23 std::vector<int> localBestBIdx(k, 0);
24 parallel_save_max(remainingPairs, localBestVal, localBestAIdx, localBestBIdx,
25 capacity, k);
26 // Reduce across all remaining pairs
27 #pragma omp parallel for num_threads(numThreads)
28 for (int i = 0; i < k; ++i)
29 {
30     #pragma omp critical
31     if (localBestVal[i] > bestValue)
32     {
33         bestValue = localBestVal[i];
34         bestAIdx = localBestAIdx[i];
35         bestBIdx = localBestBIdx[i];
36     }
37 }
38 // Reconstruct solution
39 KnapsackSolution solution{};
40 solution.totalValue = bestValue;
41 solution.totalWeight = A[bestAIdx].totalWeight + B[bestBIdx].totalWeight;
42 auto items = (A[bestAIdx] << B[bestBIdx]).getItemIndices();
43 solution.items.reserve(items.size());
44 solution.items.insert(solution.items.end(), items.begin(), items.end());
45 return solution;

```

Listato 12: Funzione knapsackcopa completa

Questa funzione mette semplicemente insieme tutti i passi dell'algoritmo: 1) si generano i 2 subsets dividendo in 2 la lista di pesi e profitti 2) si distribuiscono i blocchi tra i processori e si cercano i primi massimi profitti tra i blocchi 3) si esegue la potatura dei blocchi 4) si effettua il secondo algoritmo di ricerca dei massimi e si estrae la soluzione finale concatenando gli indici degli oggetti rappresentati come stringa binaria.

3.3 Knapsack COPA MPI

Questo algoritmo è stato concepito principalmente per un utilizzo in shared memory, non è stato pensato per essere utilizzato invece in memoria distribuita. Tuttavia è possibile farne utilizzo, introducendo delle accortezze per minimizzare il più possibile lo scambio di dati. Partendo dalle funzioni di generazione dei subsets

3.3.1 Generazione dei subsets

```
1  auto proc = compute_corank_procedure(comm, A, B);
2  int rank = comm.rank();
3  int a_start = proc.a_displacements[rank];
4  int a_end = a_start + proc.a_sizes[rank];
5  int b_start = proc.b_displacements[rank];
6  int b_end = b_start + proc.b_sizes[rank];
7  std::vector<CopaSubset> local_result(proc.output_sizes[rank]);
8  std::merge(A.begin() + a_start, A.begin() + a_end, B.begin() + b_start,
9  B.begin() + b_end, local_result.begin());
10 all_gatherv(comm, local_result.data(), proc.output_sizes[rank], output.data(),
    proc.output_sizes,
    proc.output_displacements, 0);
```

Listato 13: Funzione di merge parallelo della lista

La funzione di merging funziona calcolando parallelamente tutti i corank degli elementi, infine applica il merge su pezzi di lista distinti, esegue il gather delle varie liste usando come displacement proprio gli indici calcolati dal corank e infine distribuisce il risultato a tutti i task.

```
1  std::vector<CopaSubset> subsets{CopaSubset{}};
2  for (int item_idx = 0; const auto &item : items)
3  {
4      decltype(subsets) shifted{subsets.size()};
5      auto [w, v] = item;
6      int local_size = static_cast<int>(shifted.size()) / comm.size();
7      int remainder = static_cast<int>(shifted.size()) % comm.size();
8      auto procedure = compute_scatter_procedure(comm, subsets);
9      std::vector<CopaSubset>
10 local_shifted{static_cast<size_t>(procedure.sizes[comm.rank()]), CopaSubset{}};
11      // Distribute subsets to all processes for local shifting
12      scatterv(comm, subsets.data(), procedure.sizes, procedure.displacements,
13      local_shifted.data(),
14      procedure.sizes[comm.rank()], 0);
15      for (int i = 0; i < static_cast<int>(local_shifted.size()); i++)
16      {
17          local_shifted[i].addItem(item_idx, w, v);
18      }
19      all_gatherv(comm, local_shifted.data(), procedure.sizes[comm.rank()],
20 shifted.data(), procedure.sizes,
21 procedure.displacements, 0);
22      item_idx++;
23      std::vector<CopaSubset> newSubsets;
24      newSubsets.resize(subsets.size() + shifted.size());
25      mpi_parallel_merge(comm, subsets, shifted, newSubsets);
26      subsets = std::move(newSubsets);
27  }
28  if (reverse)
29      std::reverse(subsets);
30  return subsets;
```

Listato 14: Funzione di generazione dei subsets

La funzione di generazione funziona in modo analogo alla sua controparte shared memory, si crea una lista shifted a partire da quella presente al passo precedente, si esegue lo scattering e ogni task aggiunge internamente al suo pezzo di lista l'oggetto, infine si esegue riunisce la lista e si procede alla funzione di merging parallelo.

3.3.2 Distribuzione dei blocchi ai processori

Anche in questo caso la funzione di distribuzione si comporta in modo analogo alla versione shared memory, ogni processore prende un subrange della lista di blocchi e lo salva cercando anche l'elemento di massimo profitto. La funzione non ritorna la lista ma la usa solo per cercare il massimo.

```
1  using value_type = std::ranges::range_value_t<decltype(input)>;
2  // calculate the size of the blocks for the thread
3  int n = static_cast<int>(std::ranges::size(input));
4  int k = comm.size();
5  int i = comm.rank();
6
7  int block_size = n / k;
8  int remainder = n % k;
9
10 int start = i * block_size + std::min(i, remainder);
11 int end = start + block_size + (i < remainder ? 1 : 0);
12 auto span = std::ranges::subrange(std::ranges::begin(input) + start,
13 std::ranges::begin(input) + end);
14 int max_val = 0;
15 for (const auto &elem : span)
16 {
17     if (elem.totalValue > max_val)
18         max_val = elem.totalValue;
19 }
20 return {start, end, max_val};
```

Listato 15: Funzione di distribuzione dei blocchi

Dopodichè i processi eseguono il pruning esattamente come nella versione shared memory.

3.3.3 Funzione completa

L'aspetto della funzione di calcolo completa è analogo alla versione shared memory, la lista di pesi e profitti viene divisa a metà in 2 liste A e B. Vengono generati i subset per ognuna delle due liste e viene invertito l'ordine per la seconda. Si distribuiscono i blocchi tra i processori e poi ogni processo effettua il pruning comparando i blocchi ad esso assegnati. Finito questo insieme di operazione ogni processo avrà trovato un suo massimo, a quel punto si effettua una reduce su tutte le soluzioni trovate e si prende quella di massimo profitto.

```

1  int world = comm.size();
2  int n = static_cast<int>(weights.size());
3  std::vector<std::pair<int, int>> Alist, Blist;
4  Alist.reserve(n / 2);
5  Blist.reserve(n / 2);
6  for (int i = 0; i < n / 2; ++i)
7      Alist.emplace_back(weights[i], values[i]);
8  for (int i = n / 2; i < n; ++i)
9      Blist.emplace_back(weights[i], values[i]);
10 auto A = mpi_generate_copa_subsets(comm, Alist);
11 auto B = mpi_generate_copa_subsets(comm, Blist, true);
12 int N = static_cast<int>(B.size());
13 auto i = comm.rank();
14 // Stage 2 : Parallel suffix max for B (MaxBj and Lj)
15 CopaBlock blockA;
16 std::vector<CopaBlock> blocksB(world);
17 std::vector<CopaDistributionIndex> blockBdescriptors(world);
18 auto blockAdesc = mpi_distribute_block_per_process(comm, A);
19 auto blockBdesc = mpi_distribute_block_per_process(comm, B);
20 boost::mpi::all_gather(comm, blockBdesc, blockBdescriptors.data());
21 blockA.block = std::span(A).subspan(blockAdesc.start, blockAdesc.end -
22 blockAdesc.start);
23 blockA.maxValue = blockAdesc.maxValue;
24 for (int i = 0; i < world; ++i)
25 {
26     blocksB[i].block =
27         std::span(B).subspan(blockBdescriptors[i].start,
28 blockBdescriptors[i].end - blockBdescriptors[i].start);
29     blocksB[i].maxValue = blockBdescriptors[i].maxValue;
30 }
31 // Note: we avoid communication during distribution, each node will only
32 communicate the tasks to perform
33 std::vector<std::pair<CopaBlock, CopaBlock>> local_remaining_pairs{};
34 comm.barrier();
35 prune_block_pair(blockA, blocksB, local_remaining_pairs, capacity, i);
36 BlockPairSearchResult solution{};
37 for (const auto &[remBlockA, remBlockB] : local_remaining_pairs)
38 {
39     auto local_solution = block_pair_pointer_search(remBlockA, remBlockB,
40 capacity);
41     if (local_solution.bestVal > solution.bestVal)
42     {
43         solution = local_solution;
44     }
45 }
46 BlockPairSearchResult best{};
47 reduce(comm, solution, best, boost::mpi::maximum<BlockPairSearchResult>(), 0);
48 if (comm.rank() == 0)
49 {
50     // Reconstruct solution
51     KnapsackSolution solution{};
52     solution.totalValue = best.bestVal;
53     solution.totalWeight = A[best.bestAIdx].totalWeight +
54 B[best.bestBIdx].totalWeight;
55     //auto items = (A[best.bestAIdx] << B[best.bestBIdx]).getItemIndices();
56     solution.items.reserve(items.size());
57     solution.items.insert(solution.items.end(), items.begin(), items.end());
58     return solution;
59 }
60 return {};

```

Listato 16: Funzione completa COPA MPI

3.3.4 Considerazioni finali sull'implementazione

L'implementazione è molto sperimentale e quindi non è sicuramente ottima, ci sono diversi approcci che si possono valutare per esprimere al meglio le capacità dell'algoritmo. Il primo è sicuramente quello di usare una forma più ibrida di calcolo distribuito, in cui ogni task è una farm di thread che collaborano tra di loro in shared memory. Per rendere gli algoritmi più facilmente utilizzabili è stato creato un wrapper python mediante Boost.python, che offre la possibilità di creare classi e funzioni leggibili e eseguibili direttamente dall'interprete, la misura del tempo viene effettuata mediante il modulo time di python, che espone sostanzialmente uno steady clock, azionando l'orologio prima di invocare la funzione C++ e subito dopo alla sua conclusione, per evitare problemi con l'istanza del comunicatore MPI si delega il requisito all'eseguibile python che lo istanzia quando si include la libreria Mpi4py.

3.3.5 Strumenti e opzioni di compilazione

La libreria è stata compilata con il compilatore GCC 15.2.0 usando il set di impostazioni di default Release presente in CMake 4.2.0 (quindi -O2 e -DNDEBUG a cui si aggiungono per default -fvectorize e -ffast-math), lo standard usato è C++23 con libstdc++11, l'interprete python è python 3.12.13

4 Risultati

Prima dell'analisi dei risultati, bisogna tenere a mente alcune premesse. La prima, la più importante, è che l'algoritmo COPA proposto non può funzionare con più di 50 oggetti, questo vincolo comunque non è particolarmente oneroso per l'analisi, visto che comunque nella versione DP dell'algoritmo si può parallelizzare solo sul ciclo più interno, la cui lunghezza dipende dalla capacità. La seconda è che nel caso di un algoritmo MPI, un'implementazione puramente distribuita è sicuramente molto più inefficiente di una ibrida, che però richiederebbe degli accorgimenti ulteriori per essere implementata.

4.1 Shared Memory

i due algoritmi in shared memory hanno efficacia profondamente diversa.

4.1.1 Knapsack DP

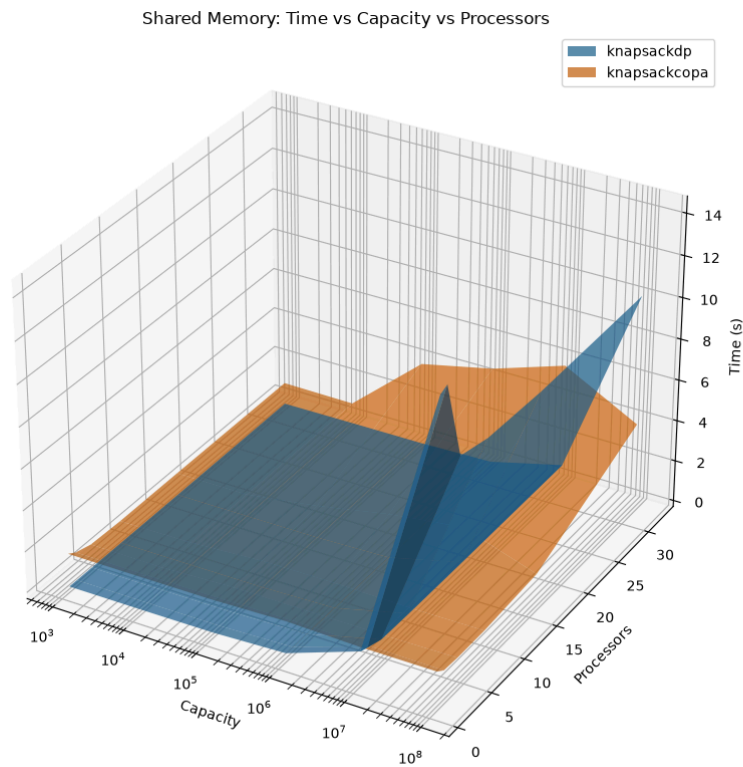


Figura 1: Confronto dei tempi di esecuzione per gli algoritmi shared memory al variare del numero di thread.

Dal grafico soprastante si osserva innanzitutto che, superata una certa capacità, i tempi di esecuzione dell'algoritmo DP crescono rapidamente, mentre quelli del COPA rimangono sostanzialmente costanti: questo è dovuto al fatto che la complessità del DP dipende dal prodotto $N \times C$ (con C la capacità dello zaino), mentre il COPA dipende solo dal numero di elementi N , generando $2^{N/2}$ sottoinsiemi indipendentemente dalla capacità.

KnapsackDP_Speedup

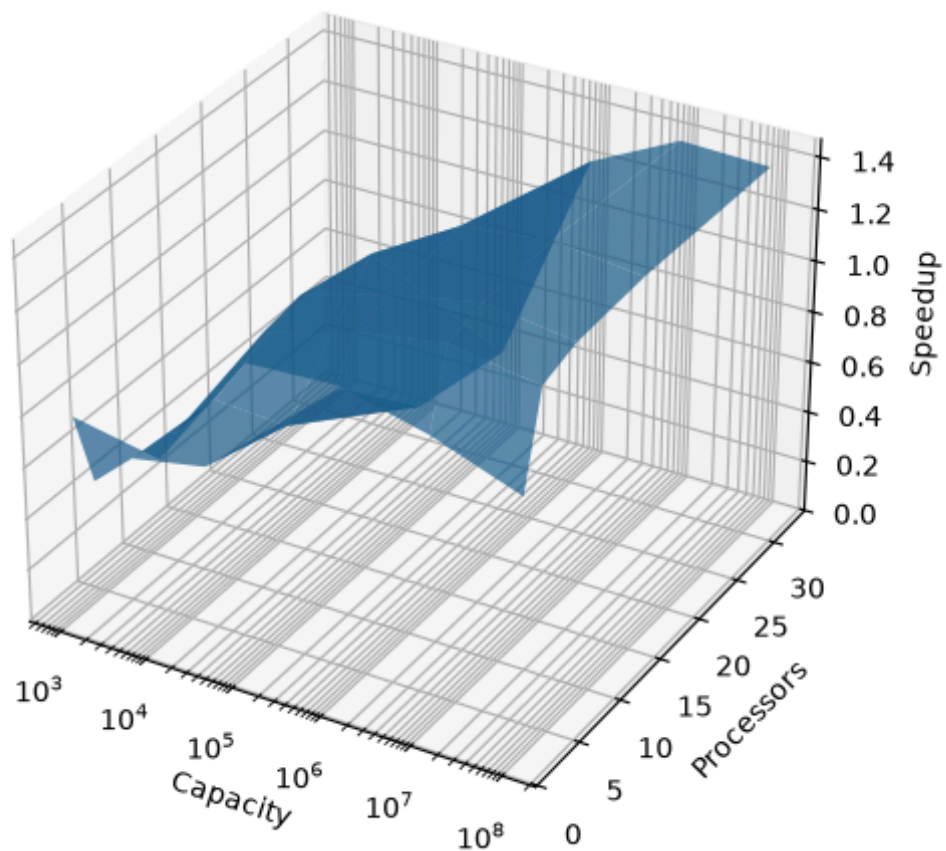


Figura 2: Speedup dell'algoritmo KPDP in shared memory

Lo speedup nel caso della versione in dynamic programming come ci si potrebbe aspettare cresce un pò con l'aumentare massiccio dei threads, questo diviene particolarmente vero quando la capacità è molto alta, soprattutto perchè a quel punto il ciclo interno sarà quello con più lavoro da eseguire. Questo comportamento smette di verificarsi quando non ci sono abbastanza elementi da poter giustificare l'istanziamento di tutti quei thread e quindi si perde di efficacia, ciò è verificabile meglio nel plot sottostante

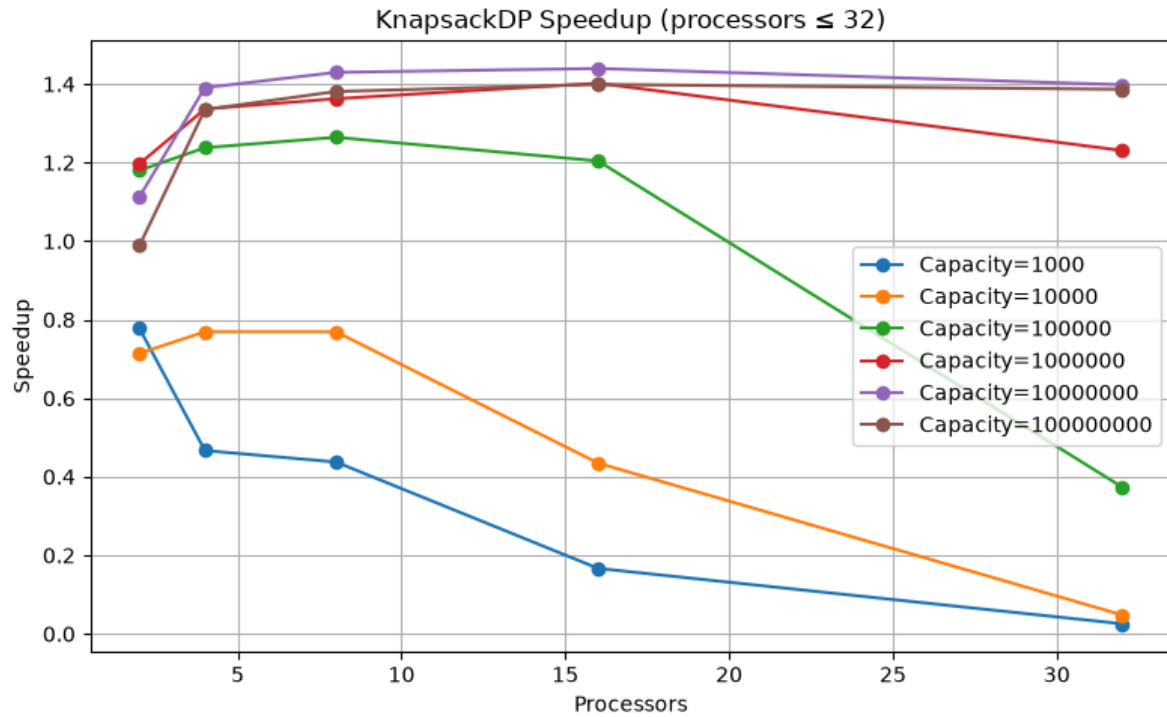


Figura 3: Speedup plot dell'algoritmo KP in DP

In questo grafico sono riportati diversi speedup a seconda della capacità, si vede subito comunque che l'algoritmo scala bene solo con i primi threads e solo quando la capacità supera una certa soglia. Diventa ancora più evidente se si osserva il grafico dell'efficienza

KnapsackDP_Efficiency

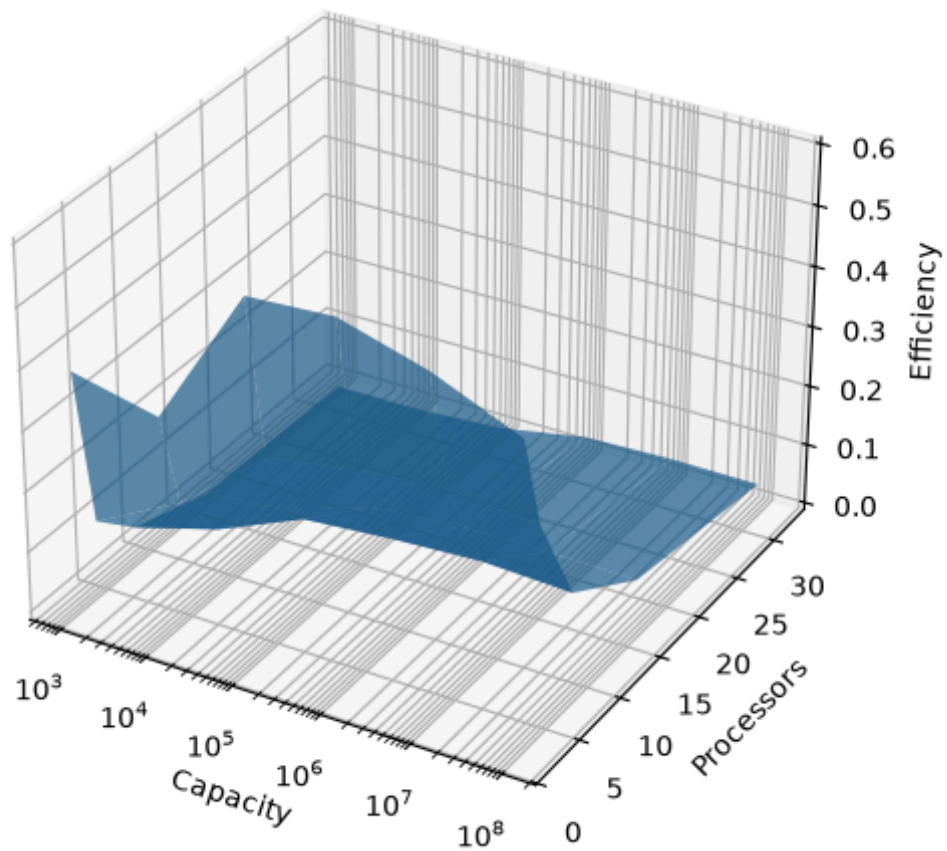


Figura 4: Efficienza dell'algoritmo KP in DP

4.1.2 Knapsack COPA

Nell'algoritmo COPA si può osservare invece che lo speedup cresce molto rapidamente all'aumentare dei processori ma decresce subito superata una certa soglia

KnapsackCOPA_Speedup

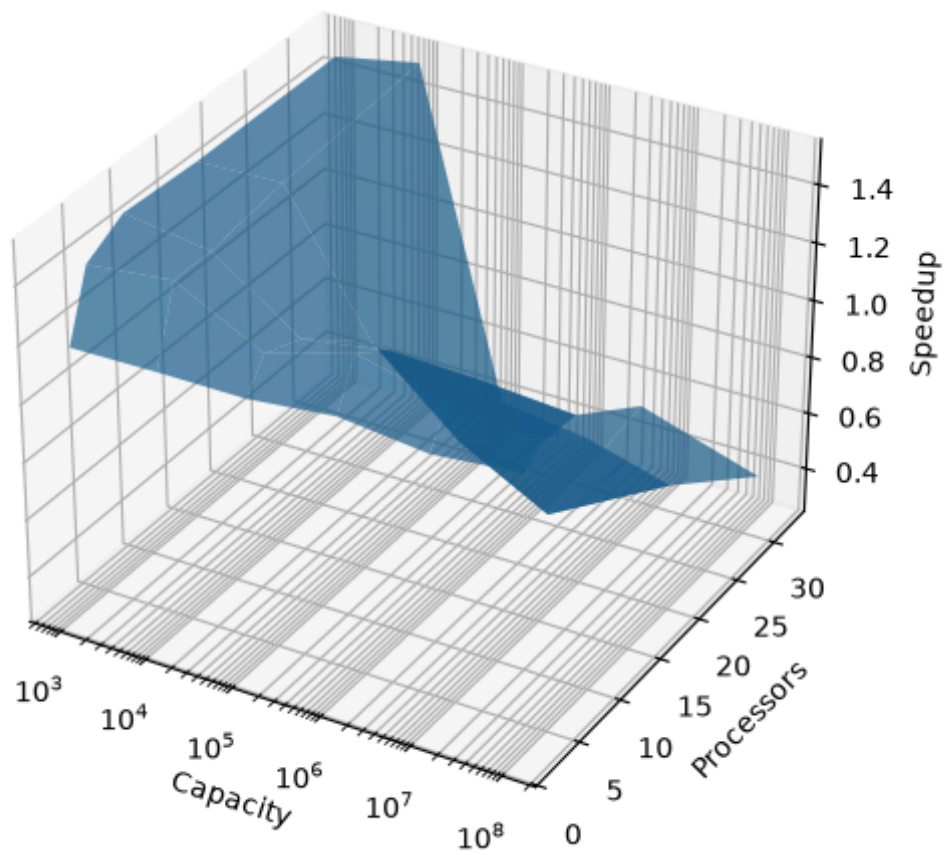


Figura 5: Speedup dell'algoritmo KP COPA in shared memory

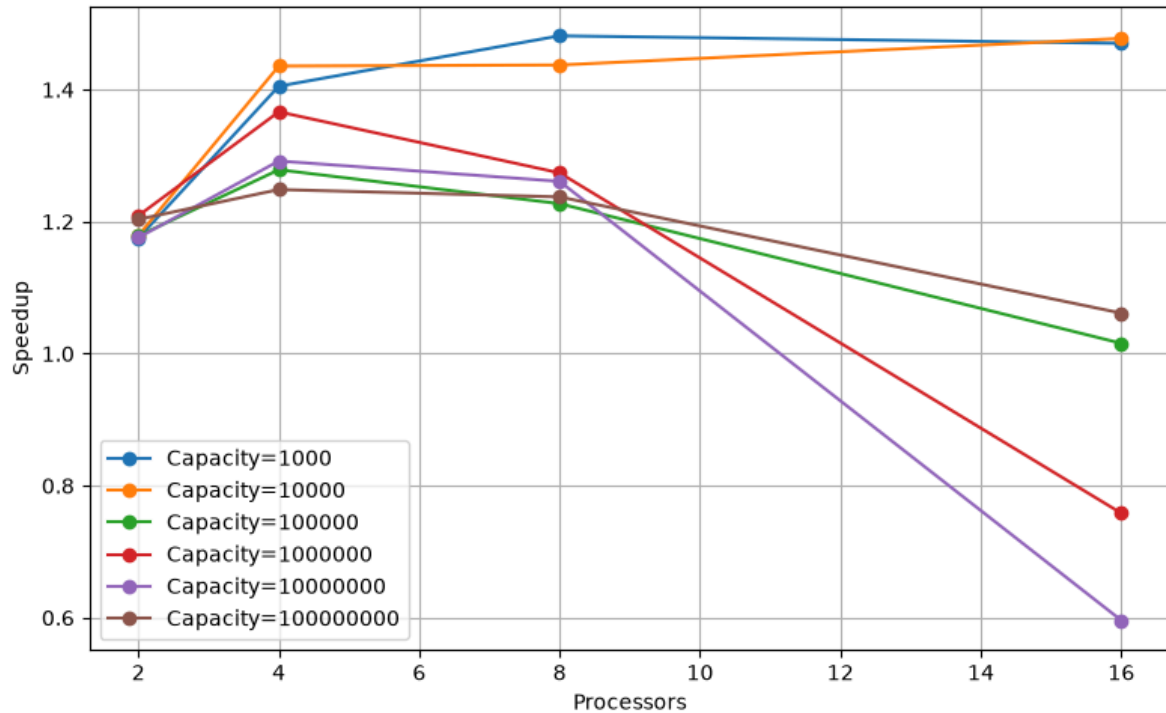


Figura 6: Speedup dell'algoritmo KP COPA in shared memory

Non bisogna sorprendersi della discesa repentina dello speedup all'aumentare dei processori, l'algoritmo presenta infatti una peculiarità: ovvero che potrebbe scalare meglio con un numero di elementi più alto, sfortunatamente il numero di soluzioni generate durante lo step di creazione dei subsets lo rende proibitivo a livello di memoria richiesta, visto che lo spazio occupato è 2^n elementi. Le linee a capacità bassa che segnano invece uno speedup di 1.4 sono perlopiù falsi positivi dovuti alla chiusura praticamente immediata del job vista l'assenza di coppie valutabili durante la fase di pruning. Ovviamente questo comportamento si riflette sull'efficienza che decresce repentinamente e sui costi che salgono rapidamente soprattutto oltre i 20 processori.

KnapsackCOPA_Cost

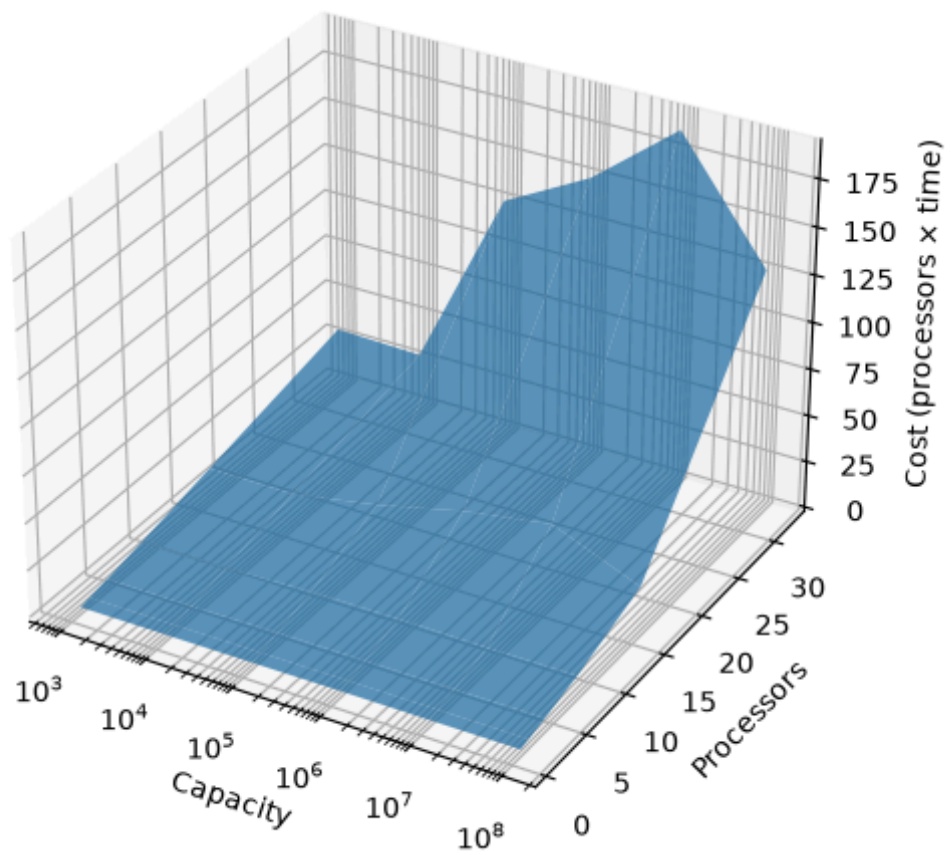


Figura 7: Costo dell'algoritmo KP COPA in shared memory

KnapsackCOPA_Efficiency

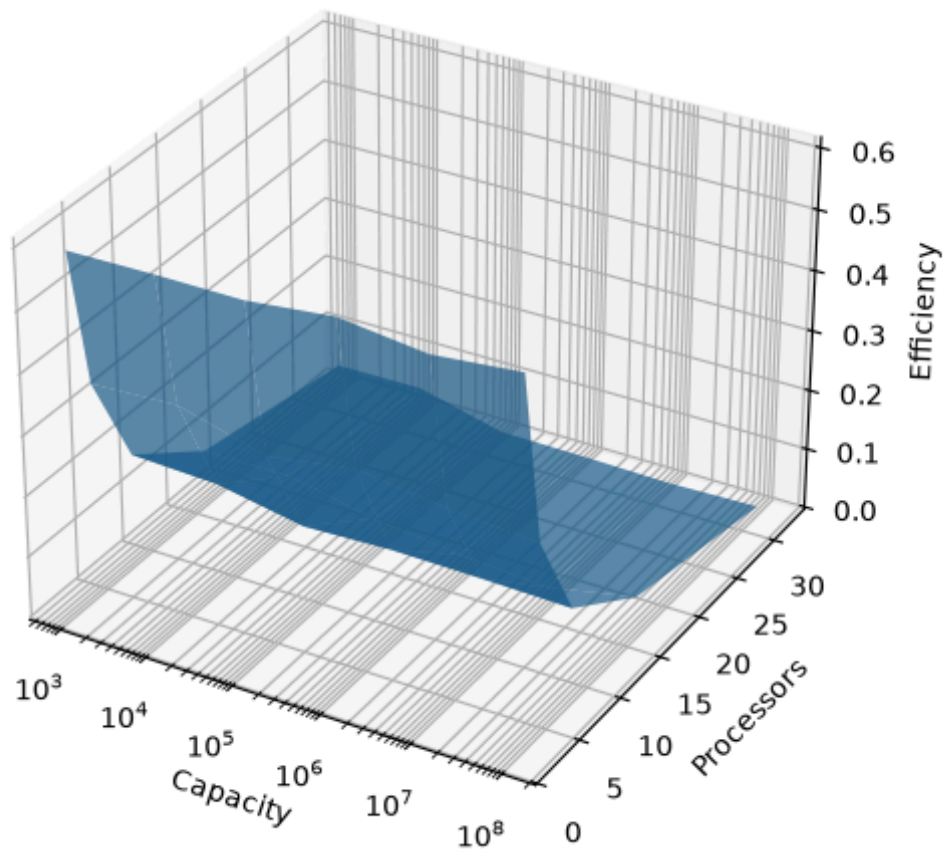


Figura 8: Efficienza dell'algoritmo KP COPA in shared memory

4.1.3 Knapsack DP vs COPA

Mettendo a confronto i due algoritmi si può osservare che, utilizzando l'algoritmo COPA nelle giuste configurazioni si possono ottenere benefici considerevoli in termini di speedup a un costo contenuto in termini di processori impiegati.

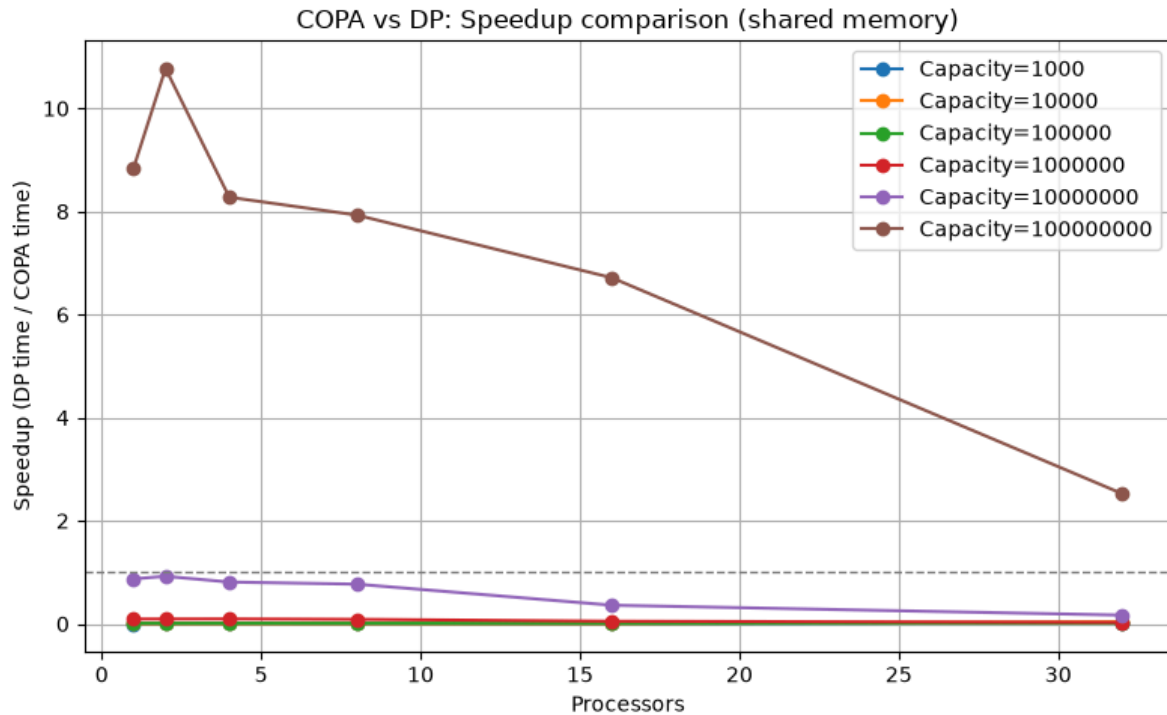


Figura 9: Speedup dell'algoritmo KP COPA vs DP

Come si può osservare dal grafico, l'algoritmo in dynamic programming vince sostanzialmente in tutte quelle configurazioni in cui la capacità è più piccola di una certa soglia, sotto il 10^7 , mentre invece per configurazioni in cui la capacità è molto grande COPA è molto più veloce. Questo conferma quanto scritto nell'articolo in cui si presenta proprio questa soluzione al problema dello zaino, va detto comunque che la limitazione d'uso a cui si è sottoposti è importante, visto che non è possibile calcolare soluzioni per insiemi più grandi di 50 elementi.

4.2 Distributed Memory

Entrambi gli algoritmi, come si può osservare nel grafico, funzionano molto male quando si utilizzano in memoria distribuita.

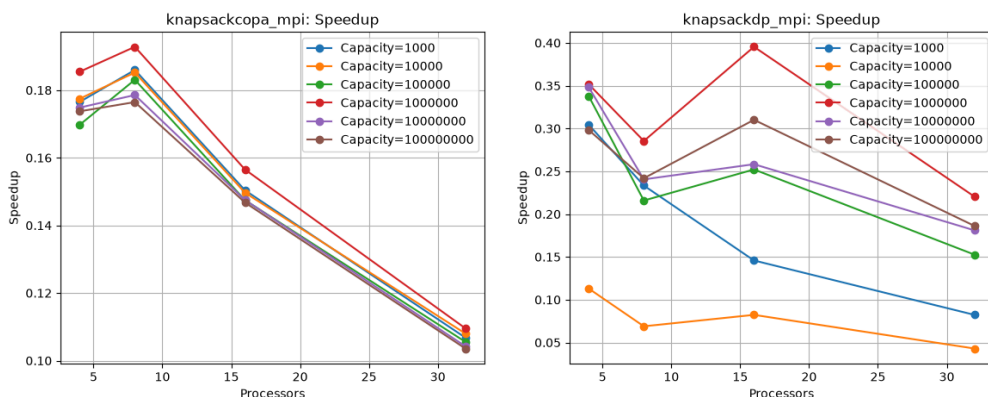


Figura 10: Speedup degli algoritmi in distributed memory

Nel caso dell'algoritmo in dynamic programming, la motivazione è semplice da individuare: il broadcast della riga ad ogni iterazione comporta un costo alto in termini di tempi di comunicazione. Supponendo di usare un intero a 32 bit, quindi 4 byte, per rappresentare il vettore di pesi e il vettore dei profitti, se si suppone una lunghezza di 10^8 elementi bisogna trasferire circa 381 Mb ogni volta solo

per il broadcast e altrettanti per eseguire il gathering e lo scattering del prossimo vettore da calcolare, rendendo questo algoritmo altamente memory bound. La versione COPA invece non presenta questo problema durante il calcolo, purtroppo durante la fase di generazione dei subsets rende il processo memory bound almeno inizialmente, visto che per ogni elemento è necessario scambiarsi tutta la lista di subsets generati, supponendo di usare 3 interi a 32 bit per rappresentare profitto, peso e indice del blocco servono $2^{n/2} * 4 * 3 * 2$ al passo finale per entrambe le liste, per cui in totale bisogna trasferire $\sum_{i=1}^n 2^{i/2} * 4 * 3 * 2$, misura che diventa considerevolmente grande una volta che il numero di elementi diventa abbastanza grande, rendendo di fatto l'algoritmo nuovamente memory bound.

4.2.1 Possibili miglioramenti

L'approccio a memoria condivisa utilizzato è molto vicino a quello ottimale, usando un dispositivo come una GPU sarebbero possibili accorgimenti ulteriori per migliorare l'efficienza: l'approccio COPA è infatti stato pensato proprio per un approccio GPU oriented, per esempio nella fase di generazione dei subset sarebbe possibile, avendo abbastanza core a disposizione, aggiungere in pochi step l'elemento corrente a tutti gli elementi della lista shifted. Con opportuni accorgimenti potrebbe essere possibile utilizzare direttamente la versione OpenMP eseguendo offloading a un dispositivo GPU, oppure si potrebbe usare OpenACC per ottenere il medesimo risultato.

Nell'approccio a memoria distribuita le modifiche possibili sono invece più sostanziose e porterebbero sicuramente a incrementi non banali in termini di prestazioni, prima di tutto l'implementazione di un approccio puramente a memoria distribuita, come si è osservato, porta inevitabilmente al dilatamento dei tempi a causa degli elevati tempi di trasmissione, inoltre nessuno di questi algoritmi è stato pensato per funzionare con un approccio a memoria distribuita ma entrambi per la memoria condivisa. Tuttavia sono comunque possibili dei miglioramenti, in entrambe le versioni invece di adottare una strategia puramente a memoria distribuita si può tentare un approccio ibrido, in cui ai task vengono date più cpu per effettuare operazioni in memoria condivisa, consentendo così di distribuire in modo più grossolano i task e evitando ritardi dovuti alle distribuzioni granulari che avvengono adesso, come i broadcast o gli all_gather utilizzati per condividere quei dati che costituiscono una dipendenza per i passi successivi dell'algoritmo in questione.

4.3 Conclusioni

Lo scopo primario del progetto era comprendere a pieno la difficoltà di implementazione dell'algoritmo e il beneficio ottenibile. L'approccio shared memory ha mostrato di essere promettente, soprattutto per quanto riguarda l'approccio COPA. Tuttavia proprio quell'approccio è assolutamente inutilizzabile nel mondo embedded, dove la memoria è una risorsa molto scarsa e quindi determinati approcci non sono possibili. Comunque se si utilizzasse invece per implementare un'ottimizzazione da terra da poi usare in orbita, potrebbe rivelarsi una scelta molto promettente per tutti quei problemi che hanno un vincolo di capacità molto elevato con valori di pesi sugli oggetti estremamente variabili tra di loro.

- [1] «Satellite Scheduling Problems: A survey of applications in Earth and outer space observation», *Science Direct*, [Online]. Disponibile su: <https://www.sciencedirect.com/science/article/pii/S0305054824003472>
- [2] «Exact and Inexact Methods for Daily Management of Earth Observation Satellite», *Harvard press*, [Online]. Disponibile su: <https://adsabs.harvard.edu/full/1996ESASP.394..507B>
- [3] K. Li, J. Liu, L. Wan, S. Yin, e K. Li, «A cost-optimal parallel algorithm for the 0–1 knapsack problem and its performance on multicore CPU and GPU implementations», *Science direct*, [Online]. Disponibile su: <https://www.sciencedirect.com/science/article/pii/S0167819115000113>